



A Just-In-Time Compilation Approach for Neural Dynamics Simulation

Chaoming Wang^{1,2}, Yingqian Jiang¹, Xinyu Liu¹, Xiaohan Lin¹,
Xiaolong Zou^{1,3}, Zilong Ji¹, and Si Wu^{1,3}(✉)

¹ School of Psychology and Cognitive Sciences, IDG/McGovern Institute for Brain Research, Peking-Tsinghua Center for Life Sciences, Academy for Advanced Interdisciplinary Studies, School of Electronics Engineering and Computer Science, Peking University, Beijing 100871, China

siwu@pku.edu.cn

² Chinese Institute for Brain Research, Beijing 102206, China

³ Beijing Academy of Artificial Intelligence, Beijing 100083, China

Abstract. As the bridge between brain science and brain-inspired computation, computational neuroscience has been attracting more and more attention from researchers in different disciplines. However, the current neural simulators based on low-level language programming or pseudo-programming using high-level descriptive language can not full fill users' basic requirements, including easy-to-learn-and-use, high flexibility, good transparency, and high-speed performance. Here, we introduce a Just-In-Time (JIT) compilation approach for neural dynamics simulation. The core idea behind the JIT approach is that any dynamical model coded with a high-level language can be just-in-time compiled into efficient machine codes running on a device of CPU or GPU. Based on the JIT approach, we develop a neural dynamics simulator in the Python framework called BrainPy, which is available publicly at <https://github.com/PKU-NIP-Lab/BrainPy>. BrainPy provides a friendly and highly flexible interface for users to define an arbitrary dynamical system, and the JIT compilation enables the defined model to run efficiently. We hope that BrainPy can serve as a general software for both research and education in computational neuroscience.

Keywords: Neural dynamics · Spiking neural networks · Neural simulator · Computational neuroscience · Just-In-Time compilation

1 Introduction

The prosperity of deep learning research in this round can be partly attributed to the popularity of machine learning software packages, such as NumPy [11], TensorFlow [1], and PyTorch [15]. These softwares based on high-level programming languages (especially Python) lower the entry barrier of normal users and

Y. Jiang and X. Liu—Equal contribution.

© Springer Nature Switzerland AG 2021

T. Mantoro et al. (Eds.): ICONIP 2021, LNCS 13110, pp. 15–26, 2021.

https://doi.org/10.1007/978-3-030-92238-2_2

greatly save their programming efforts. As the bridge between brain science and brain-inspired computation, computational neuroscience aims to use mathematical models to elucidate the principles of brain functions, and thus inspire the development of artificial intelligence. Nowadays, computational neuroscience is becoming more and more important, whose development needs more researchers coming from different disciplines, such as biology, mathematics, physics etc. However, compared to the deep learning community, a general programming framework based on a high-level language for neural dynamics simulation is still lacking.

Currently, most tools for simulating neural dynamics provide two types of methods: low-level programming or descriptive language. The representative frameworks in the first category include NEURON [6], NEST [9], GENESIS [3], and CARLsim [7]. These simulators offer a library of standard models and the corresponding convenient interface based on Python. Users can create networks in Python by grouping the models in the library. However, once a new model is needed, the user must learn to program using a low-level language, such as SLI in NEST, C++ in CARLsim, and NMODL in NEURON, which increases the learning cost of the user dramatically and restricts the flexibility of the simulator to define new models [21]. Another approach for model definition is code generation based on a descriptive language [2]. The representative frameworks employing this approach include Brian2 [18,19], ANNarchy [22], Brain Modeling ToolKit [8], GeNN [23], NESTML [16], NeuroML [5,10], and NineML [17]. These simulators allow users to create new models based on descriptions, such as text, JSON or XML files, in high-level programming languages (like Python). Afterwards, high-level model descriptions are translated into a low-level code to speed up the running. The code generation approach has made great success as it enables rapid prototyping for conventional models. However, its coding flexibility is limited by the anticipation of possible models. Once the variety of models goes beyond the anticipation, extension to accommodate new components must be made in both high-level and low-level languages. This is hard or nearly impossible for normal users. Moreover, the generation procedure and the generated code are usually hidden, making the logic controlling, model debugging, and model error correction not transparent to users. Generally speaking, the code generation approach has intrinsic limitations on flexibility and transparency. This is because the coding based on descriptive languages is not real programming where users can freely define variables and express their thoughts.

Motivated by solving the foregoing shortcomings, we introduce a Just-In-Time (JIT) compilation approach for neural dynamics simulation and develop a simulator based on Python called **BrainPy**. *Easy-to-learn-and-use* is important for most users starting to program models in computational neuroscience. Rooted in Python, BrainPy provides a user-friendly interface to program dynamical models. Compared to the frameworks based on low-level programming languages, a BrainPy user only needs to learn the NumPy array programming [11] and the basic Python syntax, such as the if-else condition and the for-loop control. *Flexibility* in model building is increasingly important for a researcher in computational neuroscience, as new models and concepts emerge constantly in the field. Different from simulators based on descriptive languages, BrainPy

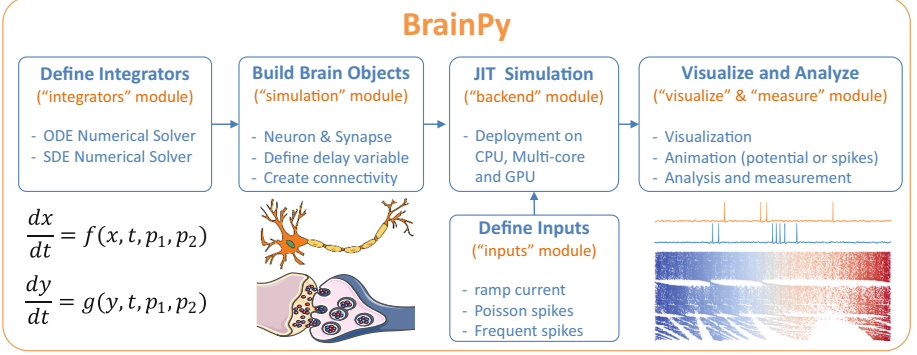


Fig. 1. The overall workflow of the simulator **BrainPy**.

endows users with the full data/logic flow control. No assumption is imposed on the model form. Any dynamical model can be coded in a Python class with inheritance from the base **DynamicSystem**. In this way, the *performance* can also be guaranteed. Because the child class of **DynamicSystem** in **BrainPy** can be just-in-time compiled into efficient machine codes running on a device of CPU or GPU.

2 Results

2.1 The Overview of the Framework

The overall workflow of **BrainPy** consists of four stages: (1) defining numerical integrators for differential equations, (2) building brain objects for simulation, (3) running models with the JIT acceleration and (4) analyzing and visualizing the simulation results (Fig. 1). In **BrainPy**, the first stage involves defining the numerical integrators for all differential equations. This step is important but very easy, since **BrainPy** provides a simple and intuitive way to help the user. The next stage involves defining all the wanted brain models with **NeuGroup**, **TwoEndConn** and **Network**. These three objects are children classes of **DynamicSystem**. In this step, **BrainPy** provides a highly flexible way which allows users to define any variables and update logic. The third stage is accomplished with a single function named `run(duration, inputs)`, where `duration` refers to the start and end time points of each simulation, and `inputs` specifies the input data to model variables. At this stage, **BrainPy** gathers all defined neurons and connections across neuron groups, and just-in-time compiles them into machine codes, and then deploys the codes to the target hardware specified by the user. At the final state, the user can perform various analyses and visualizations of the simulation output.

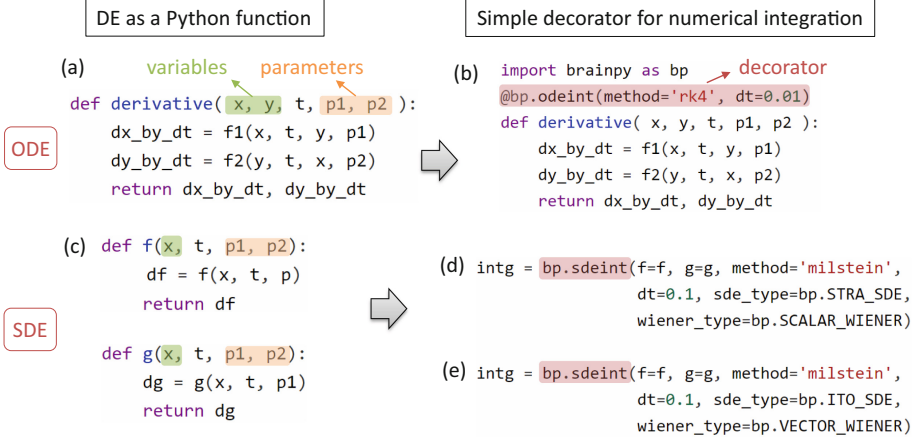


Fig. 2. Numerical solvers provided in BrainPy for ODEs and SDEs.

2.2 Stage 1: Defining Numerical Solvers

The essence of neural dynamics simulation is to solve differential equations. BrainPy provides various numerical solvers for ordinary differential equations (ODEs) and stochastic differential equations (SDEs). Specifically, BrainPy supports explicit Runge-Kutta numerical methods and the Exponential Euler method for ODE and SDE numerical integration. It also supports a variety of adaptive Runge-Kutta methods for ODEs.

In BrainPy, any differential equation can be defined as a Python function. For example, for a two-dimensional ODE system,

$$\frac{dx}{dt} = f_1(x, t, y, p_1), \quad (1a)$$

$$\frac{dy}{dt} = f_2(y, t, x, p_2), \quad (1b)$$

we can define it as a function illustrated in Fig. 2(a), where t denotes the time variable, p_1 and p_2 after t denote parameters needed in this system, and x and y before t denote dynamical variables. In the function body, the derivative for each variable can be customized by the user's need (f_1 and f_2). After defining the derivative function, numerical integration can be implemented with a simple decorator `@odeint(method, dt)` (Fig. 2(b)), where `method` denotes the numerical method used to integrate the ODE function, and `dt` controls the numerical integration precision.

Same as the ODE system, any SDE system expressed as

$$dx = f(x, t, p_1)dt + g(x, t, p_2)dW, \quad (2)$$

can be defined as two Python functions as shown in Fig. 2(c). Specifically, the drift and diffusion coefficients in the SDE system are coded as f and g functions.

For the SDE function with a scalar noise, the size of the return data df and dg should be the same, e.g., $df \in R^d, dg \in R^d$. However, for a more general SDE system, it usually has multi-dimensional driving Wiener processes:

$$dx = f(x, t, p_1)dt + \sum_{\alpha=1}^m g_{\alpha}(x, t, p_2)dW^{\alpha}. \quad (3)$$

For such a m -dimensional noise system, the coding schema is the same as the scalar one, except that the return size of dg is multi-dimensional, e.g., $df \in R^d, dg \in R^{d \times m}$. SDEs have two ways of integral: Itô and Stratonovich stochastic integrals [14], and BrainPy supports both of them. Specifically, the numerical integration of SDEs are performed by `sdeint(f, g, method, dt, wiener_type, sde_type)` (Fig. 2(d) and Fig. 2(e)), where `wiener_type` denotes the type of Wiener process (`SCALAR.WIENER` or `VECTOR.WIENER`) and `sde_type` the integral type (`ITO_SDE` or `STRA_SDE`).

2.3 Stage 2: Building Brain Objects

BrainPy is designed to facilitate flexible model customization. A set of pre-defined models about neurons, synapses, connectivities and networks can not allow us to capture the full range of possible model logic. Alternatively, we should provide a concise but productive and expressive interface to help users code new models with the logic even never seen before. In BrainPy, this step can be implemented by object-oriented programming with the inheritance from the base class `DynamicSystem`.

Generally speaking, any dynamical system can evolve continuously or discontinuously. Discontinuous evolution may be triggered by events, such as the reset of membrane potential, the pre-synaptic spike, the intracellular calcium concentration, or the firing rate intensity. Moreover, it is common in a neural system that a dynamical system has different states, such as the excitable or refractory state in a leaky integrate-and-fire (LIF) model. In this section, we will illustrate how to use two children classes of `DynamicSystem` (specifically, `NeuGroup` and `TwoEndConn`) to capture these complexity in dynamics modeling.

LIF model has the lowest computation cost but is difficult to implement, since it has a continuous potential evolution and a discontinuous reset operation, along with a refractory period. Specifically, it is given by,

$$\tau \frac{dV(t)}{dt} = -(V(t) - V_r) + RI(t), \quad (4a)$$

$$\text{if } V(t) \geq V_{th}, V(t) \leftarrow V_r, \text{ and last } \tau_{ref} \text{ ms}, \quad (4b)$$

where V is the membrane potential, τ the membrane time constant, and R the membrane resistance. Whenever V reaches a fixed threshold V_{th} (i.e., $V \geq V_{th}$), the neuron generates a spike and its potential is reset to the rest value V_r , followed by the refractory period τ_{ref} . In BrainPy, the built-in class `NeuGroup` can be used to describe the kinetics of a population of neurons.

```

1 import brainpy as bp
2 import numpy as np
3
4 class LIF(bp.NeuGroup):
5     @staticmethod
6     # integrate V with Exponential Euler method
7     @bp.odeint(method='exponential_euler')
8     def int_V(V, t, Iext, Vr, R, tau):
9         dV = (- (V - Vr) + R * Iext) / tau
10        return dV
11
12    def __init__(self, num, **kwargs):
13        super(LIF, self).__init__(size=num, **kwargs)
14        # parameters
15        self.Vr, self.Vth = -60, -50
16        self.tau, self.R, self.t_ref = 20., 1., 5.
17        # variables
18        self.V = np.ones(num) * self.Vr # potential variable
19        self.t_last_spike = np.ones(num) * -1e7 # last spike time
20        self.spikes = np.zeros(num, dtype=bool) # spike state
21        self.ref = np.zeros(num, dtype=bool) # refractory state
22        self.I = np.zeros(num) # external inputs
23
24    def update(self, _t, _i):
25        for i in range(self.num):
26            # reset the spike STATE
27            self.spikes[i] = False
28            # reset the refractory STATE
29            self.ref[i] = True
30            # if neuron i is not in refractory STATE
31            if _t - self.t_last_spike[i] > self.t_ref:
32                V = self.int_V(self.V[i], _t, self.I[i],
33                               self.Vr, self.R, self.tau)
34                if V >= self.Vth: # if generate a spike EVENT
35                    self.V[i] = self.Vr
36                    self.spikes[i] = True
37                    self.t_last_spike[i] = _t
38            else:
39                self.V[i] = V
40                self.ref[i] = False
41        self.I[i] = 0. # reset the input variable

```

Listing 1.1. Simulation code for a population of LIF neurons.

The above code snippet (Listing 1.1) demonstrates the creation of a LIF neuron group by inheriting from `NeuGroup`. The model parameters and variables are initialized in `__init__` function, the continuous potential evolution is defined in `int_V` function with the help of `odeint` decorator, and the state switch and event processing in `update` function are intuitively coded with the basic Python if-else condition and for-loop syntax.

Different from the neuron modeling, synaptic models usually need to (1) create connectivity between neuronal populations and (2) process the transmission delay from the source to the target neurons. BrainPy provides `TwoEndConn` to

capture these features. We illustrate this with the exponential synapse, whose kinetics is given by:

$$\frac{ds(t)}{dt} = -\frac{s(t)}{\tau} + \sum_k \delta(t - t^k), \quad (5a)$$

$$I(t) = g_{max}s(t - D)(V(t) - E), \quad (5b)$$

with s the synaptic state, τ the decay time, t^k the pre-synaptic spike time, D the synaptic delay, V the post-synaptic potential, g_{max} the synaptic conductance, and E the reversal potential.

```

1 class ExpSyn(bp.TwoEndConn):
2     @staticmethod
3     # integrate s with Exponential Euler method
4     @bp.odeint(method='exponential_euler')
5     def int_s(s, t, tau):
6         ds = - s / tau
7         return ds
8
9     def __init__(self, pre, post, conn, tau,
10                  g_max, E, delay=0., **kwargs):
11         # parameters
12         self.tau, self.g_max, self.E = tau, g_max, E
13         # connections
14         self.conn = conn(pre.size, post.size)
15         self.pre_ids, self.post_ids = \
16             conn.requires('pre_ids', 'post_ids')
17         self.num = len(self.pre_ids)
18         # variables
19         self.s = bp.ops.zeros(self.num) # synaptic state
20         # the delayed synaptic state
21         self.g=self.register_constant_delay('g', self.num, delay)
22         # initialize the base class
23         super(ExpSyn,self).__init__(pre=pre, post=post, **kwargs)
24
25     def update(self, _t):
26         # continuous synaptic state evolution
27         self.s = self.int_s(self.s, _t, self.tau)
28         # traverse each connected neuron pair
29         for i in range(self.num):
30             pre_i, post_i = self.pre_ids[i], self.post_ids[i]
31             # update synaptic state by the pre-synaptic spike
32             if self.pre.spikes[pre_i]: # if has spike EVENT
33                 self.s[i] += 1.
34             # push synaptic state to delay
35             self.g.push(i, self.s[i])
36             # output currents to the post-synaptic neuron
37             g = self.g.pull(i) * self.g_max # get delayed state
38             self.post.I[post_i] += g*(self.E-self.post.V[post_i])

```

Listing 1.2. Simulation code for exponential synapses between two populations.

Listing 1.2 shows the implementation of the exponential synapse model. Compared with the neuron model coding in Listing 1.1, there are two major differences. First, the synaptic connections `self.pre_ids` and `self.post_ids` are constructed by the instance of `Connector` (`conn`, see Listing 1.3). Second, synaptic delays are registered with the function call by `self.register_constant_delay`. Such registered delay variables will be automatically updated in simulation.

To simulate a neural system, we usually need to group a large number of neuron populations as well as synaptic connections between them to form a network. `Network` in BrainPy thus is provided to manage this burdensome task. Users only need instantiate the customized models and then pack them into the `Network` container. The syntax parsing, code compilation, and device deployment etc. will then be automatically processed. For example, the following Python code implements a conductance-based E/I balanced network (COBA-LIF) [4].

```

1 # excitatory neuron group
2 E = LIF(3200, monitors=['spike'])
3 E.V = np.random.randn(E.num) * 5 + E.Vr
4 # inhibitory neuron group
5 I = LIF(800, monitors=['spike'])
6 I.V = np.random.randn(I.num) * 5 + I.Vr
7 # excitatory synapses from E group to E group
8 E2E = ExpSyn(pre=E, post=E, conn=bp.connect.FixedProb(0.02),
9             tau=5., g_max=0.6, E=0.)
10 # excitatory synapses from E group to I group
11 E2I = ExpSyn(pre=E, post=I, conn=bp.connect.FixedProb(0.02),
12             tau=5., g_max=0.6, E=0.)
13 # inhibitory synapses from I group to E group
14 I2E = ExpSyn(pre=I, post=E, conn=bp.connect.FixedProb(0.02),
15             tau=10., g_max=6.7, E=-80.)
16 # inhibitory synapses from I group to I group
17 I2I = ExpSyn(pre=I, post=I, conn=bp.connect.FixedProb(0.02),
18             tau=10., g_max=6.7, E=-80.)
19 # pack neurons and synapses into a network
20 net = bp.Network(E, I, E2E, E2I, I2E, I2I)

```

Listing 1.3. Simulation code for a conductance-based E/I balanced network model.

2.4 Stage 3: Simulation with JIT Acceleration

The magic of `Network` is not limited to the convenient brain object packing, but it also provides the JIT compilation for the defined model.

Python is known highly convenient but simultaneously very slow. This is because all variables in Python are dynamically typed. In order to accelerate the running speed, many JIT compilers, such as PyPy [20], Pyston[13] and Numba [12], have been introduced in Python. Especially, in the Python scientific computing ecosystem, Numba has been a standard for JIT compilation, in which Python functions can be just-in-time compiled to either CPU machine codes using industry-standard LLVM compiler library or GPU codes using NVIDIA PTX compiler. Thus, it may be very suitable for JIT compilation in neural dynamics simulation, and there is no need to create another JIT compiler by

ourself. Unfortunately, Numba's JIT compilation works best for functions, not Python classes. In order to make BrainPy's object-oriented programming be compatible with Numba's functional programming, we build a simple Abstract Syntax Trees (AST) based compiler to automatically translate users' classes into functions. Simply state, all the data and functions accessed by `self`. will be interpreted as the function arguments in our AST compiler. In such a way, coding in BrainPy not only keeps the advantages of object-oriented programming, but also obtains the running efficiency of JIT compilation. In practice, users can firstly configure the simulation backend by calling:

```
1 # set simulation backend to single-threaded CPU
2 bp.backend.set(jit=True, device='cpu')
3
4 # set simulation backend to paralleled CPU
5 bp.backend.set(jit=True, device='multi-cpu')
6
7 # set simulation backend to paralleled GPU
8 bp.backend.set(jit=True, device='cuda')
```

Then, the simulation is started by a single functional call:

```
1 # start a simulation
2 net.run(duration=100., # simulation time length
3         inputs=[(E, 'I', 20.), # inputs to E.I variable
4                 (I, 'I', 20.)], # inputs to I.I variable
5         report=True) # turn on the progress report
```

in which `inputs` can be used to mimic the patch clamp or transcranial magnetic stimulation in a physiological experiment.

2.5 Stage 4: Analysis and Visualization of Simulation Output

To gain insight from neural dynamics simulation, it is necessary to further analyze and visualize the simulation output. BrainPy provides useful analyzing functions and visualization tools in `measure` and `visualize` modules, respectively. `measure` module implements a wide range of analyzing functions commonly used in neuroscience (like `cross_correlation()` and `firing_rate()`). Moreover, generic plotting functions in `visualize` module, e.g., `line_plot()`, `animate_1D()` and `animate_2D()`, are able to statically or interactively display spikes, membrane potentials, etc. For example, a single line of code can effectively present the spiking results after simulation in the above stages:

```
1 bp.visualize.raster_plot(E.mon.ts, E.mon.spikes, show=True)
```

The visualization output of the above function call is shown in Fig. 3.

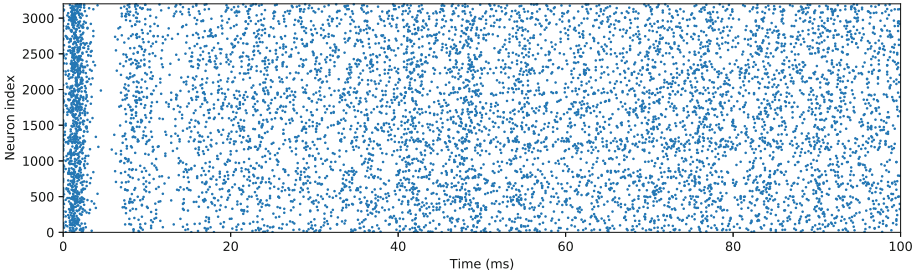


Fig. 3. The raster plot of the COBA-LIF network model in a 100-ms run.

3 Comparison of Running Efficiency

A key factor in neural dynamics simulation is the running efficiency. To compare this with currently popular neural simulators, we adopt two benchmark network models [4]: a COBA-LIF network and a Hodgkin–Huxley model based network (COBA-HH). We implemented two benchmarks on Brian2, NEST and NEURON simulators. All experiments had the duration of several seconds. Dynamical equations were solved using the Exponential Euler method with a step size of 0.1 ms. All simulations were run on Python 3.8 installed on Ubuntu 20.04.2 LTS, with Intel(R) Core(TM) i7-6700K CPU @ 4.00 GHz.

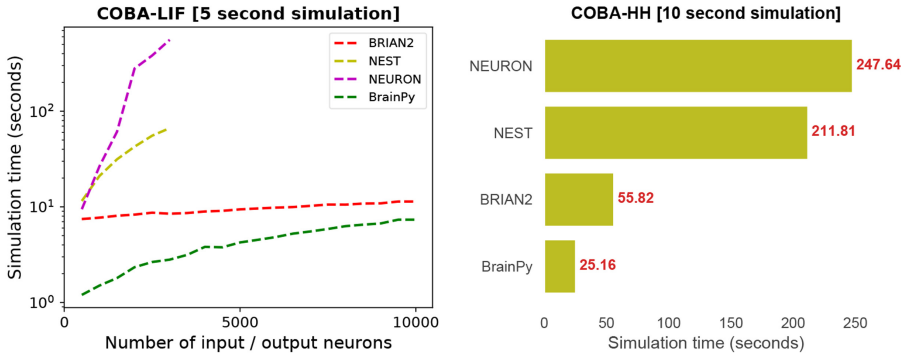


Fig. 4. Speed comparison between NEURON, NEST, Brian2 and BrainPy. (a) The network-size scaling test based on the COBA-LIF network. The x-axis denotes the number of neurons and the y-axis the (logarithmic) simulation time. (b) The speed comparison on the COBA-HH network.

As shown in Fig. 4, due to the JIT compilation, BrainPy has the best performance on both benchmarks. Notably, NEST and NEURON are significantly slower than other simulators, but this may only demonstrate that they are not suitable to simulate point neuron models, as NEURON is designed to simulate biological-detailed multi-compartment models, while NEST has focused on

simulating large-scale network models on distributed computing systems. Nevertheless, it demonstrates that BrainPy is efficient for simulating medial-size networks with simplified neurons.

4 Conclusion

We have developed a neural simulator **BrainPy**, which exploits the JIT compilation approach for neural dynamics simulation. We demonstrated that BrainPy can effectively satisfy the needs of users in computational neuroscience. First, by providing a concise and intuitive interface for neural modeling, BrainPy is *easy to learn and use*. Second, with the minimal assumption and restriction in the design, BrainPy endows the user with the full *flexibility and transparency* to define arbitrary dynamical systems. Third, by incorporating the JIT compilation, BrainPy achieves highly *efficient* dynamics simulation. Overall, BrainPy exhibits significant advantages on neural dynamics simulation, as the incorporated JIT approach preserves the intrinsic productive and expressive power of Python programming, and meanwhile realizes efficient running with the speed approaching to that of C or FORTRAN. We hope that BrainPy can serve as a general platform to support users programming efficiently in computational neuroscience. BrainPy is open source and can be available at <https://github.com/PKU-NIP-Lab/BrainPy>.

References

1. Abadi, M., et al.: Tensorflow: large-scale machine learning on heterogeneous distributed systems (2016). arXiv preprint [arXiv:1603.04467](https://arxiv.org/abs/1603.04467)
2. Blundell, I., et al.: Code generation in computational neuroscience: a review of tools and techniques. *Front. Neuroinf.* **12**, 68 (2018)
3. Bower, J.M., Beeman, D.: *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural Simulation System*. Springer, Heidelberg (2012). <https://doi.org/10.1007/978-1-4612-1634-6>
4. Brette, R., et al.: Simulation of networks of spiking neurons: a review of tools and strategies. *J. Comput. Neurosci.* **23**(3), 349–398 (2007)
5. Cannon, R.C., et al.: Lems: a language for expressing complex biological models in concise and hierarchical form and its use in underpinning neuroml 2. *Front. Neuroinf.* **8**, 79 (2014)
6. Carnevale, N.T., Hines, M.L.: *The NEURON Book*. Cambridge University Press, Cambridge (2006)
7. Chou, T.S., et al.: Carlsim 4: an open source library for large scale, biologically detailed spiking neural network simulation using heterogeneous clusters. In: 2018 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE (2018)
8. Dai, K., et al.: Brain modeling toolkit: an open source software suite for multiscale modeling of brain circuits. *PLOS Comput. Biol.* **16**(11), e1008386 (2020)
9. Gewaltig, M.O., Diesmann, M.: Nest (neural simulation tool). *Scholarpedia* **2**(4), 1430 (2007)

10. Gleeson, P., et al.: Neuroml: a language for describing data driven models of neurons and networks with a high degree of biological detail. *PLoS Comput. Biol.* **6**(6), e1000815 (2010)
11. Harris, C.R., et al.: Array programming with numpy. *Nature* **585**(7825), 357–362 (2020)
12. Lam, S.K., Pitrou, A., Seibert, S.: Numba: a llvm-based python jit compiler. In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6 (2015)
13. Modzelewski, K., Wachtler, M., Galindo, P.: Pyston (2021). <https://github.com/pyston/pyston>
14. Øksendal, B.: *Stochastic Differential Equations: An Introduction with Applications*. Springer, Heidelberg (2003). <https://doi.org/10.1007/978-3-642-14394-6>
15. Paszke, A., et al.: Pytorch: an imperative style, high-performance deep learning library. In: Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., Garnett, R. (eds.) *Advances in Neural Information Processing Systems* 32, pp. 8024–8035. Curran Associates, Inc. (2019). <https://www.pytorch.org/>
16. Plotnikov, D., Rumpe, B., Blundell, I., Ippen, T., Eppler, J.M., Morrison, A.: Nestml: a modeling language for spiking neurons (2016). arXiv preprint [arXiv:1606.02882](https://arxiv.org/abs/1606.02882)
17. Raikov, I., et al.: Nineml: the network interchange for neuroscience modeling language. *BMC Neurosci.* **12**(1), 1–2 (2011)
18. Stimberg, M., Brette, R., Goodman, D.F.: Brian 2, an intuitive and efficient neural simulator. *Elife* **8**, e47314 (2019)
19. Stimberg, M., Goodman, D.F., Benichoux, V., Brette, R.: Equation-oriented specification of neural models for simulations. *Front. Neuroinf* **8**, 6 (2014)
20. Team, T.P.: (2019). <https://www.pypy.org/>
21. Tikidji-Hamburyan, R.A., Narayana, V., Bozkus, Z., El-Ghazawi, T.A.: Software for brain network simulations: a comparative study. *Front. Neuroinf.* **11**, 46 (2017)
22. Vitay, J., Dinkelbach, H.Ü., Hamker, F.H.: Annarchy: a code generation approach to neural simulations on parallel hardware. *Front. Neuroinf.* **9**, 19 (2015)
23. Yavuz, E., Turner, J., Nowotny, T.: Genn: a code generation framework for accelerated brain simulations. *Sci. Rep.* **6**(1), 1–14 (2016)